Azure data Factory

BootCamp_Project-1

Student Name: Bhaskar Karri

Program: Azure Data Engineering

Table of Contents

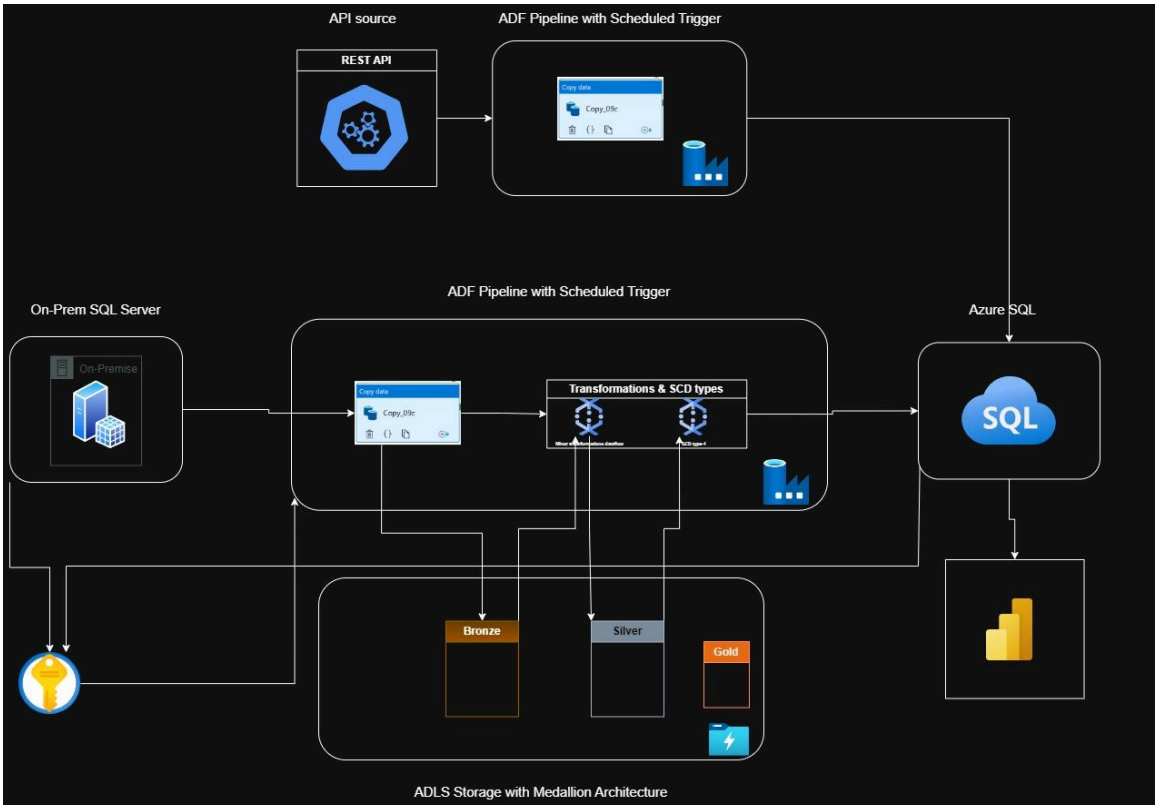
BootCamp_Project-1 ..... **1**
1. Project Overview **2**
2. Architecture **2**
3. Technologies Used **3**
4. Source System **3**
Example Table Schema 3
5. Medallion Architecture **4**
Bronze Layer (Raw Data) 4
Silver Layer (Clean Data) 5
Gold Layer (Curated Data) 6
6. Azure Data Factory Pipelines **6**
Pipeline 1: RESTAPI → Azure SQL 7
Pipeline 2: On-Prem SQL → Bronze Layer 7
7. Data Transformation (Silver Layer) **8**
8. Slowly Changing Dimensions (SCD) **9**
SCD Type 1 Implementation 9
10. Data Validation **11**
11. Security Implementation **11**
12. Triggers **12**
13. Power BI Reporting **12**
14. Challenges Faced **12**
15. Key Implementations **13**
16. Run the Pipeline: **14**
17. Conclusion **15**

1. Project Overview

This project demonstrates an end-to-end modern data engineering pipeline using Azure services. Data is extracted from a REST API and On-Premises SQL Server database, ingested into Azure Data Lake Storage Gen2, transformed using Azure Data Factory Mapping Data Flows, and finally loaded into Azure SQL Database for analytics and reporting.

The pipeline follows the Medallion Architecture, consisting of Bronze, Silver, and Gold layers to ensure scalable, maintainable, and high-quality data processing.

2. Architecture



RESTAPI → Azure Data Factory → Azure SQL

On-Prem SQL → Azure Data Lake → Azure SQL → Power BI

3. Technologies Used

Technology	Purpose
Azure Data Factory (ADF)	Data orchestration and pipeline automation
Azure Data Lake Storage Gen2	Data storage
Mapping Data Flows	Data transformation and SCD type1
Azure SQL Database	Data warehouse layer
Power BI	Visualization
Azure Key Vault	Secure credential management

4. Source System

Source1: REST API https://jsonplaceholder.typicode.com/

Source2: On-Premises SQL Server database containing the following tables:

Customer

Product

ProductCategory

Sales

Example Table Schema

Customer

Results | Messages

	ORDINAL_POSITION	COLUMN_NAME	DATA_TYPE
1	1	CustomerID	int
2	2	CustomerName	varchar
3	3	City	varchar
4	4	State	varchar
5	5	Email	varchar

Product

Results | Messages

	ORDINAL_POSITION	COLUMN_NAME	DATA_TYPE
1	1	ProductID	int
2	2	ProductName	varchar
3	3	CategoryID	int
4	4	Price	decimal

ProductCategory

Results | Messages

	ORDINAL_POSITION	COLUMN_NAME	DATA_TYPE
1	1	CategoryID	int
2	2	CategoryName	varchar

Sales

Results | Messages

	ORDINAL_POSITION	COLUMN_NAME	DATA_TYPE
1	1	SalesID	int
2	2	CustomerID	int
3	3	ProductID	int
4	4	Quantity	int
5	5	SalesDate	date
6	6	TotalAmount	decimal

5. Medallion Architecture

Bronze Layer (Raw Data)

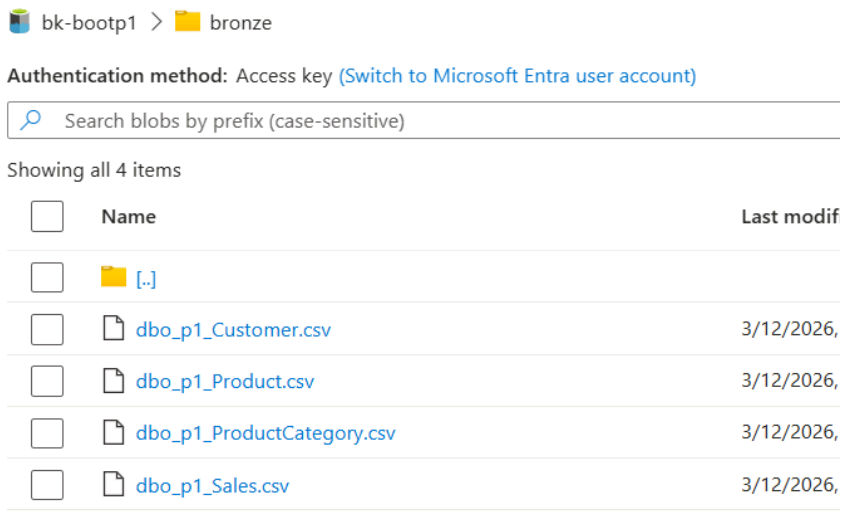
Purpose:

Store raw data exactly as received from source.

Characteristics:

- No transformations
- Historical storage
- Data stored in CSV format

Structure:



ADF pipeline uses Copy Activity to ingest data from On-Prem SQL to Bronze.

Silver Layer (Clean Data)

Purpose:

Store cleaned and validated data.

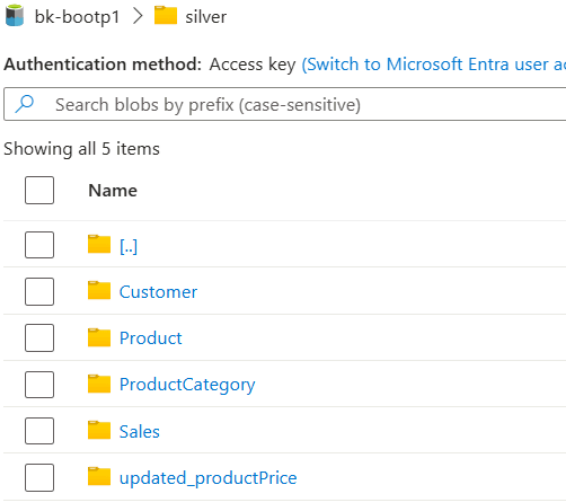
Transformations performed:

1. Remove duplicate records
2. Validate Primary Keys
3. Remove NULL or zero values
4. Remove unnecessary columns
5. Standardize schema

Format used:

Parquet

Structure:



ADF Mapping Data Flow performs minor transformations and copy the data in silver layer

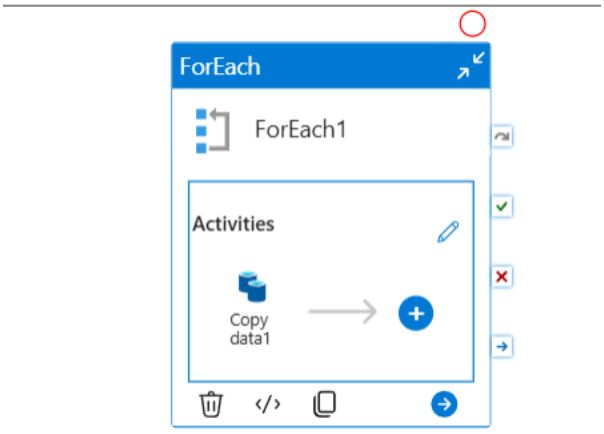
Gold Layer (Curated Data)

Purpose:

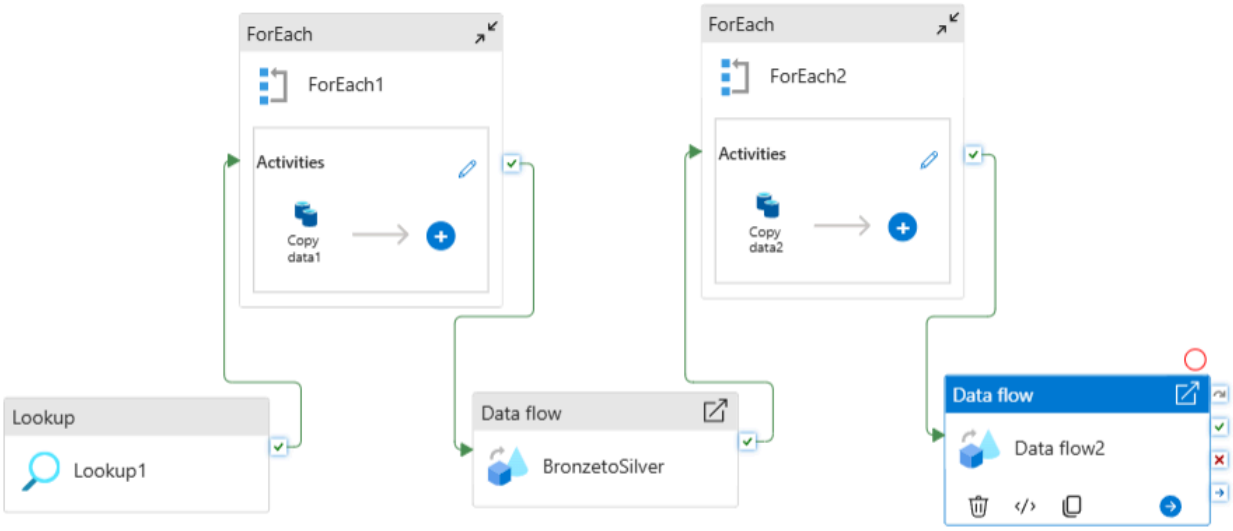
Provide analytics-ready data models. Can be used to store final copy of data in parquet.

6. Azure Data Factory Pipelines

Pipeline1: PL_Copy_API_AzureSQL



Pipeline2: PL_ONPremSQL_to_AzureSQL_SCDtype1



Pipeline 1: RESTAPI → Azure SQL

Steps:

1. Pipeline starts.
2. ForEach reads API endpoint list.
3. Each iteration calls a different API endpoint.
4. Data is retrieved as JSON.
5. Copy activity loads data into the corresponding SQL table.

Parameters used:

Base_url: https://jsonplaceholder.typicode.com/

Relative_urls :["/posts","/comments","/albums","/photos","/todos","/users"]

ForEach Expression: `@pipeline().parameters.relative_urls`

Pipeline 2: On-Prem SQL → Bronze Layer

Steps:

1. Configure Self Hosted Integration Runtime
2. Create Linked Service for On-Prem SQL

3. Create Linked Service for ADLS Gen2
4. Use Copy Activity

Process:

SQL Tables → CSV Files in Bronze Layer

Query used while lookup: `SELECT *`

```
FROM ADE_trainingdb.INFORMATION_SCHEMA.TABLES

WHERE TABLE_TYPE = 'BASE TABLE'

AND TABLE_NAME IN
('p1_Customer','p1_ProductCategory','p1_Product','p1_Sales');
```

Foreach activity: `@activity('Lookup1').output.value`

Source: onpremsqlcopy_ds

Sink: adls_csv

7. Data Transformation (Silver Layer)

ADF Mapping Data Flow performs transformations.

Transformations Implemented

1. Source (Bronze CSV)
2. Filter
3. Select
4. Derived Column
5. Sink (Silver Parquet)

Validation checks:

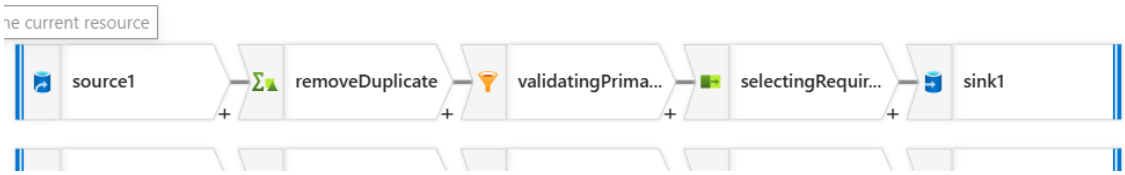
Primary Key should not be NULL

Primary Key should not be zero

Remove duplicate records

Remove unused columns

Data flow used: DF_Bronze_To_Silver

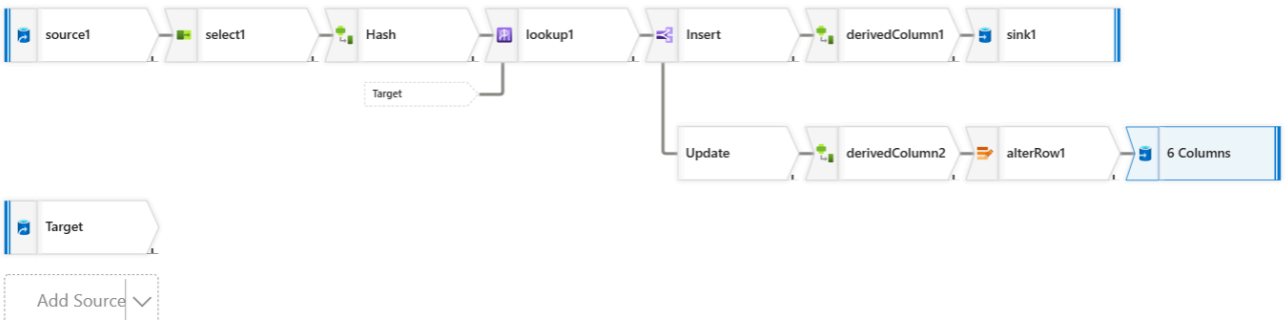


Source: Bronze layer

Sink: Silver Layer

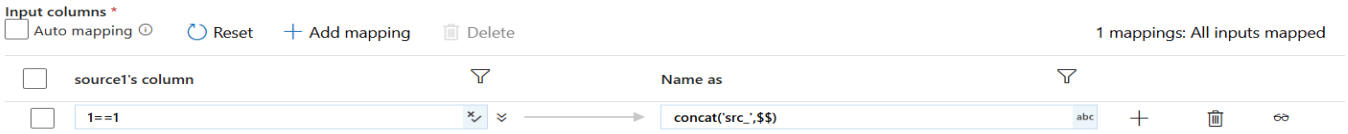
8. Slowly Changing Dimensions (SCD)

SCD Type 1 Implementation



Source: bk-bootp1/silver/Customer

Select:



Creating Hash Column:

```
toBinary(
    crc32(
        concat(
            toString(src_CustomerID),
            iifNull(src_CustomerName,'NA'),
```

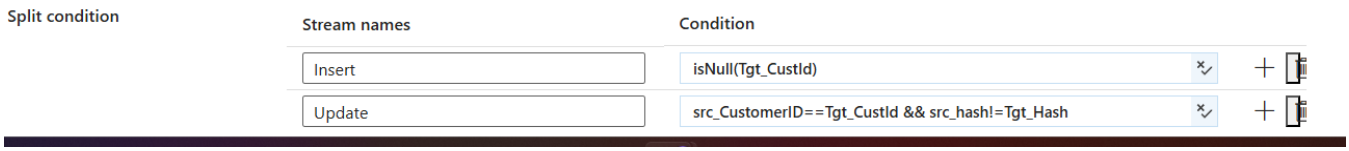
```
            iifNull(src_City,'NA'),
            iifNull(src_State,'NA'),
            iifNull(src_Email,'NA')
        )
    )
)
```

Reference target source: SELECT [CustomerID] AS Tgt_CustId,[Tgt_Hash] FROM [dbo].[Customer]

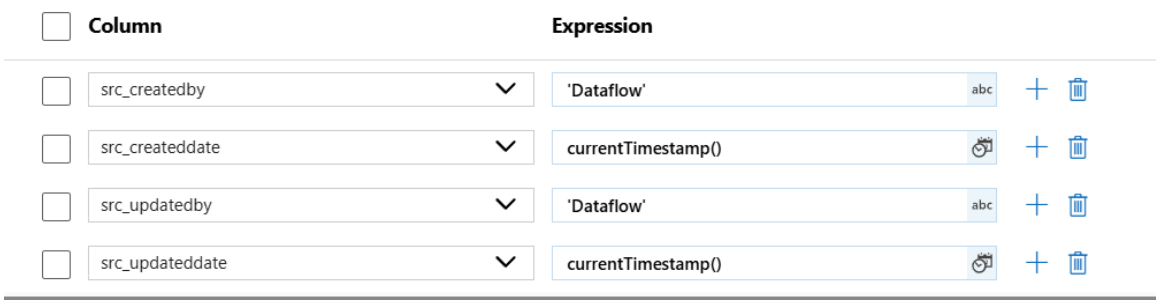
From Azure SQL

Lookup:

Split condition:Inserting

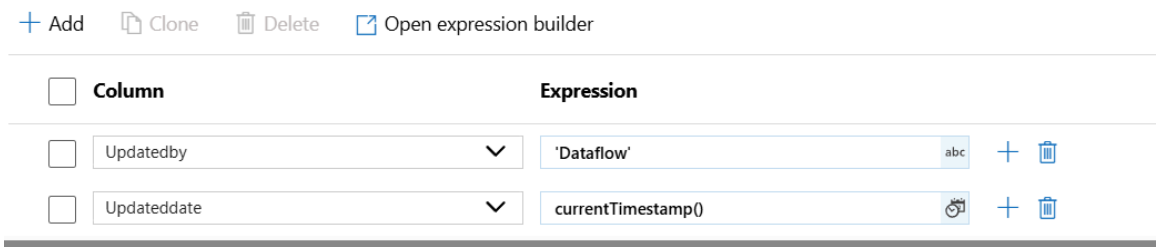


Derived Columns:

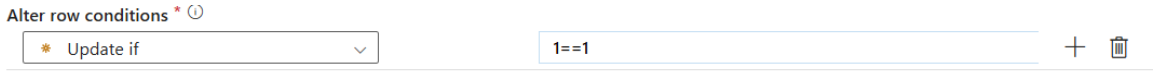


Sink: Azure SQL Database

Updating:



Altering row:



10. Data Validation

Several validation checks were implemented:

Row Count Validation

Bronze Row Count = Silver Row Count

Silver Row Count = Gold Row Count

Primary Key Validation

Ensure:

CustomerID IS NOT NULL

CustomerID != 0

11. Security Implementation

Azure Key Vault used to store:

Database Password

Connection Strings

ADF Linked Service retrieves credentials securely.

12. Triggers

Two triggers were configured:

Scheduled Trigger

Runs ingestion pipelines periodically.

Event Trigger

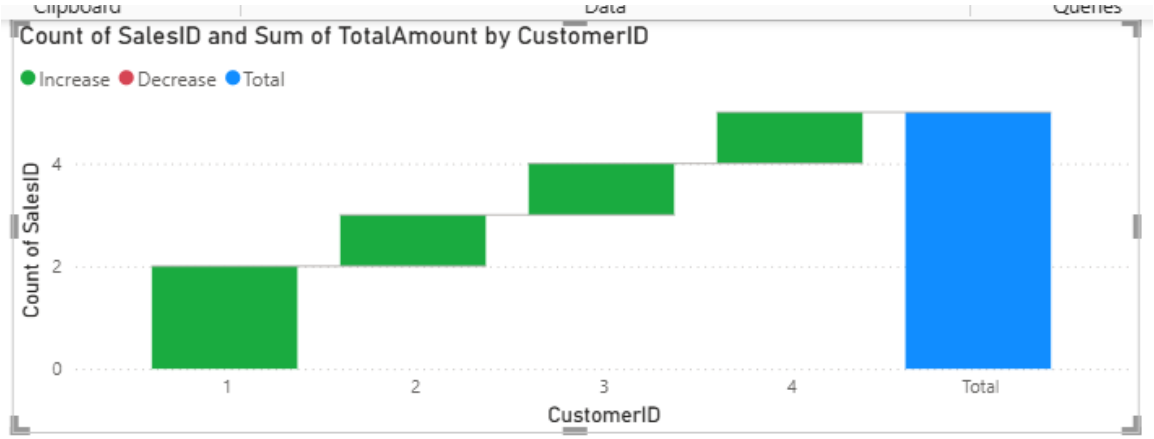
Used for Silver layer processing when new files arrive.

13. Power BI Reporting

Final curated data in Azure SQL Database is used for reporting.

Power BI connects using:

Azure SQL Database connector



14. Challenges Faced

- During the implementation of this project, several challenges were encountered while building and running the data pipelines.
- One of the major challenges was handling schema mismatches between Parquet files and Azure SQL tables. In some cases, the column names in the Parquet files generated from the Silver layer did not match the column names defined in the SQL tables, which caused errors during the data load process. This issue was resolved by carefully validating the schema and configuring the correct column mappings in the Copy Activity.

- Another challenge involved handling duplicate records when loading data into Azure SQL Database. Since some tables had primary key constraints, the pipeline failed when duplicate records were inserted. This problem was addressed by adjusting the loading strategy and ensuring the tables were either truncated before loading or configured with appropriate update logic.
- A further issue occurred due to foreign key constraint violations, especially while loading the Sales table. The Sales table referenced Product and Customer tables, and when the Sales data was loaded before the related dimension tables, the pipeline failed. This was resolved by modifying the pipeline execution order so that dimension tables such as Product and Customer were loaded before the Sales table.
- There were also challenges related to file path configuration in Azure Data Lake Storage. Incorrect folder paths or missing wildcard file names caused the pipeline to fail when reading Parquet files. This issue was fixed by configuring the correct dataset paths and using wildcard patterns to read multiple Parquet files generated by the Data Flow.
- Another difficulty encountered was while implementing Slowly Changing Dimensions (SCD) using Azure Data Factory Mapping Data Flows. Configuring the Lookup and Alter Row transformations correctly required careful comparison between source and existing records to determine whether a record should be inserted or updated. After debugging the transformation logic, the SCD Type 1 and Type 2 implementations worked as expected.
- Finally, configuring connectivity between the On-Premises SQL Server and Azure Data Factory was initially challenging due to network and integration runtime setup issues. This was resolved by installing and configuring the Self-Hosted Integration Runtime to securely connect the on-premises environment with Azure services.
- Overall, these challenges provided valuable experience in debugging pipelines, handling schema inconsistencies, managing data dependencies, and implementing best practices for building reliable data engineering solutions in Azure.

15. Key Implementations

- Implemented **Medallion Architecture**
- Built **dynamic pipelines in ADF**
- Implemented **SCD Type 1**
- Integrated **On-Prem SQL with Azure**
- Performed **data validation and transformations**

16. Run the Pipeline:

- Pipeline triggers as per the schedule

In order to run the pipeline manually:

- Prepare Source Database
  Ensure the On-Premises SQL Server database is running and the required tables (Customer, Product, ProductCategory, and Sales) are created with sample data.
- Start Self-Hosted Integration Runtime
  Verify that the Self-Hosted Integration Runtime used to connect the On-Prem SQL Server with Azure Data Factory is installed and running.
- Verify Linked Services
  Check that all Linked Services in Azure Data Factory are configured correctly:
  - On-Prem SQL Server
  - Azure Data Lake Storage Gen2
  - Azure SQL Database
  - Azure Key Vault (if used)
- Run the Bronze Ingestion Pipeline
  Execute the pipeline that copies data from the On-Prem SQL Server to the Bronze layer in Azure Data Lake Storage. This step stores the raw data in CSV format.
- Run the Silver Transformation Pipeline
  Execute the Data Flow pipeline that reads data from the Bronze layer, performs data cleaning and validation, and writes the transformed data to the Silver layer in Parquet format.
- Load Data into Azure SQL Database
  Run the Copy Activity pipeline that loads the curated data from the Gold or Silver layer into the corresponding tables in Azure SQL Database.
- Validate Data
  Verify that the data has been loaded successfully by checking the tables in Azure SQL Database and comparing row counts with the source data.
- Connect Power BI (Optional)
  Open Power BI Desktop and connect to Azure SQL Database to create reports and dashboards for analyzing the processed data.

17. Conclusion

This project demonstrates how to build a scalable, modern data pipeline using Azure Data Engineering tools. The implementation ensures data quality, maintains historical changes using SCD techniques, and enables business analytics through Power BI dashboards.